Assignment 2, Part A: Evidence

All outputs below were captured from the running stack (`docker compose up -d --build api`, API at `http://localhost:8000`).

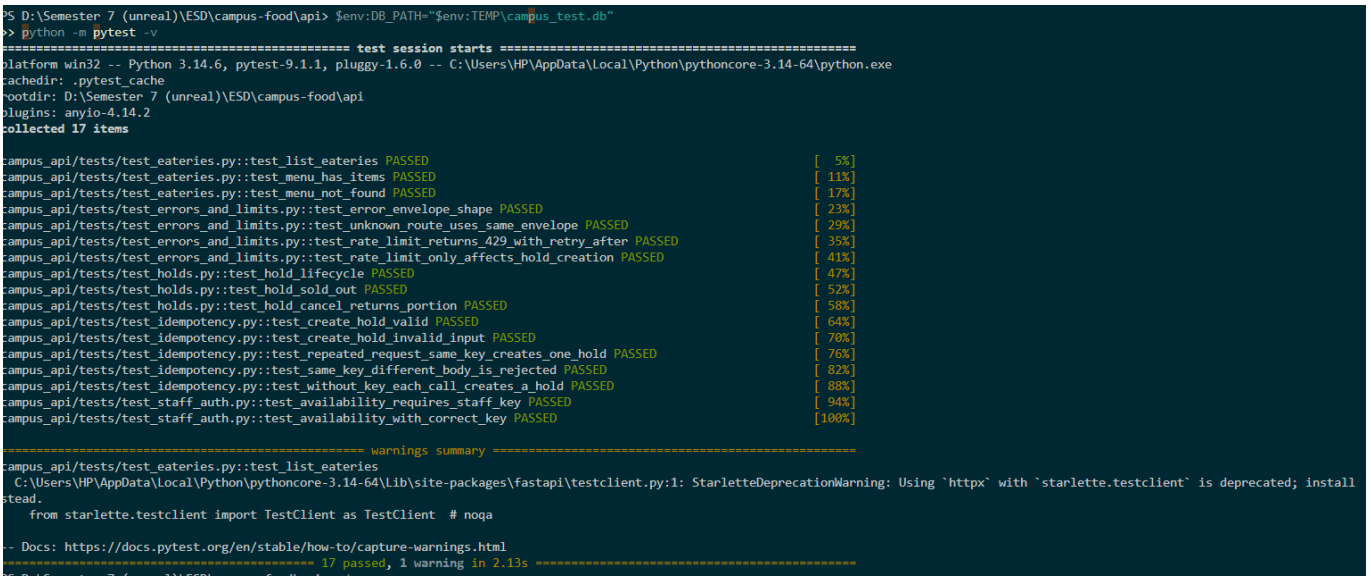
---

1. Automated tests

Command (from `api/`):

```
$env:DB_PATH="$env:TEMP\campus_test.db"
python -m pytest -v
```

Result: all tests passed. The only message was a Starlette deprecation warning about `httpx` in the test client, which does not affect results.



Tests cover:

Requirement	Test
Valid request	`test_create_hold_valid`
Invalid input	`test_create_hold_invalid_input`
Repeated request	`test_repeated_request_same_key_creates_one_hold`
Reused key, different body	`test_same_key_different_body_is_rejected`
Error envelope	`test_error_envelope_shape`, `test_unknown_route_uses_same_envelope`
Rate limit	`test_rate_limit_returns_429_with_retry_after`, `test_rate_limit_only_affects_hold_creation`

---

2. Consistent error format

Every error uses `{"error": {"code", "message", "request_id", "details"?}}`. The `request_id` in the body equals the `x-request-id` response header, which is the same ID written to the JSON logs.

404, unknown hold

```
$ curl.exe -i localhost:8000/holds/9999
HTTP/1.1 404 Not Found
content-type: application/json
x-request-id: 72542927ffbd49aa

{"error":{"code":"HOLD_NOT_FOUND","message":"hold not
found","request_id":"72542927ffbd49aa"}}
```

422, invalid input (`item_id` must be greater than 0)

```
$ curl.exe -i -X POST localhost:8000/holds -H "Content-Type: application/json" -d
'{"item_id": 0}'
HTTP/1.1 422 Unprocessable Entity
content-type: application/json
x-request-id: 04795cd4ae924385

{"error":{"code":"VALIDATION_ERROR","message":"request validation
failed","request_id":"04795cd4ae924385","details":
[{"field":"item_id","message":"Input should be greater than 0"}]}}
```

404, unknown route (same envelope, not FastAPI's default body)

```
$ curl.exe -i localhost:8000/nope
HTTP/1.1 404 Not Found
content-type: application/json
x-request-id: 919f426ba1154157

{"error":{"code":"NOT_FOUND","message":"Not
Found","request_id":"919f426ba1154157"}}
```

Link between response and logs

The 404 response above carried `request_id 72542927ffbd49aa` (also in the `x-request-id` header). Searching Kibana (data view `Campus food logs`, time range covering 20 Oct 2026) for `app.request_id: "72542927ffbd49aa"` returns exactly one log entry from the API container:

Field	Value
`@timestamp`	`2026-10-20T19:01:02.397Z`

Field	Value
app.request_id	72542927ffbd49aa
app.event	request_completed
app.method / app.path	GET / /holds/9999
app.route	/holds/{hold_id}
app.status	404
app.severity	WARNING
app.duration_ms	2.8
container.name	campus-app
_index	campus-food-logs-2026.10.20

The raw log line (`event.original`) that the API wrote to stdout and Filebeat shipped:


```
{"time": "2026-10-20T19:01:02.397Z", "service": "campus-food-api", "severity":
"WARNING", "message": "GET /holds/{hold_id} -> 404", "request_id":
"72542927ffbd49aa", "event": "request_completed", "method": "GET", "path":
"/holds/9999", "route": "/holds/{hold_id}", "status": 404, "duration_ms": 2.8}
```


So the ID the client receives in the error body is the same ID that appears in the logs, which lets an error report be traced to its log line. The log records the route template (`/holds/{hold_id}`) as well as the real path, which keeps the metric/log label bounded.

Screenshot: `![alt text](kibana_request.png)` (Kibana Discover with this entry expanded).

---

3. Idempotency (`Idempotency-Key` on `POST /holds`)

The same request was sent twice with `Idempotency-Key: demo-1`.

First call: creates the hold

```
HTTP/1.1 201 Created
x-request-id: 91fcb66d2c304895

{"id":248,"item_id":1,"eatery":"main-
cafeteria","pickup_code":"0593","status":"active","planned_pickup_minutes":180}
```

Second call: replays the stored response

```
HTTP/1.1 201 Created
idempotent-replayed: true
```

```
x-request-id: 6055c4a7550646e7

{"id":248,"item_id":1,"eatery":"main-
cafeteria","pickup_code":"0593","status":"active","planned_pickup_minutes":180}
```

Same hold id (248) and pickup code, plus the `idempotent-replayed: true` header, so no second hold was created.

How it works: the key and a hash of the request body are stored in the `idempotency_keys` table in the same transaction as the hold. The same key with a different body returns `422 IDEMPOTENCY_KEY_REUSED`. `BEGIN IMMEDIATE` makes two simultaneous retries queue up instead of both creating a hold.

Other operations:

- `PATCH /items/{id}/availability` sets an absolute value, so repeating it gives the same result.
- `GET` endpoints are safe to repeat.
- `pickup` and `cancel` return `409 HOLD_NOT_ACTIVE` on a repeat and change nothing.

---

4. Rate limiting (`POST /holds`)

Limit: 5 requests per 10 seconds, sliding window, in memory (single instance). **Shared by:** all callers with the same client IP (inside Docker, host requests often share the gateway IP). `X-Forwarded-For` is not trusted because a client can fake it. **Allowed again:** when the oldest counted request leaves the 10 s window, which is the number of seconds in `Retry-After`. Rejected requests are not counted, so retrying does not extend the block.

Accepted requests, then 429

```
request 1 -> 201
request 2 -> 201
request 3 -> 201
request 4 -> 201
request 5 -> 201
request 6 -> 429
request 7 -> 429
```

The 429 response

```
HTTP/1.1 429 Too Many Requests
retry-after: 10
content-type: application/json
x-request-id: 433156db6b5e48dc

{"error":{"code":"RATE_LIMITED","message":"too many requests, retry in
10s","request_id":"433156db6b5e48dc"}}
```

Allowed again after waiting 10 seconds

```
HTTP/1.1 201 Created
x-request-id: 878f9e0bf58249e7

{"id":254,"item_id":4,"eatery":"main-
cafeteria","pickup_code":"8854","status":"active","planned_pickup_minutes":180}
```

The limiter logs each rejection (`event: rate_limited`), alongside the normal request log:

```
{"time": "2026-10-20T19:02:53.346Z", "service": "campus-food-api", "severity":
"WARNING", "message": "rate limit hit, retry in 10s", "request_id":
"433156db6b5e48dc", "event": "rate_limited", "client": "172.19.0.1",
"retry_after": 10}
```

---

5. Python caller (`caller/client.py`)

Settings

Setting	Value
Connect timeout	1 s
Read timeout	2 s
Retry limit	3 attempts per call (2 retries)
Backoff	0.5 s, then 1 s, plus up to 0.1 s random jitter
Retried on	timeouts, connection errors, 502/503/504
Never retried	4xx responses; `POST` without an `Idempotency-Key`
Breaker opens after	3 consecutive failed attempts
Breaker recovery period	10 s, then one half-open trial call

`POST /holds` is retried only because the caller sends one `Idempotency-Key` per logical operation and reuses it on every retry.

Failure used: the A1 fault switch `menu_delay_ms=3000` (longer than the 2 s read timeout), set and cleared through `/admin/faults`. Rate limiting is demonstrated separately in section 4.

Output

```
00:03:56 === Phase 1: healthy server
00:03:56 --- menu read 1
00:03:56 GET /eateries/1/menu -> 200 (attempt 1)
00:03:56     result: 200, 4 items
```

```
00:03:56 --- menu read 2
00:03:56 GET /eateries/1/menu -> 200 (attempt 1)
00:03:56     result: 200, 4 items
00:03:56 --- menu read 3
00:03:56 GET /eateries/1/menu -> 200 (attempt 1)
00:03:56     result: 200, 4 items
00:03:56 POST /holds -> 201 (attempt 1)
00:03:56     hold created: id=255 (Idempotency-Key 06f800cd...)
00:03:56 === Phase 2: inject 3000 ms delay (read timeout is 2.0s)
00:03:56 [demo] menu delay set to 3000 ms
00:03:56 --- menu read (expect timeouts and retries)
00:03:58 GET /eateries/1/menu attempt 1/3 failed: ReadTimeout
00:03:58   retrying in 0.52s
00:04:01 GET /eateries/1/menu attempt 2/3 failed: ReadTimeout
00:04:01   retrying in 1.08s
00:04:04 [breaker] closed -> open
00:04:04 GET /eateries/1/menu attempt 3/3 failed: ReadTimeout
00:04:04     result: FAILED (GET /eateries/1/menu failed)
00:04:04 --- menu read (expect: blocked by open breaker)
00:04:04 GET /eateries/1/menu BLOCKED: breaker is open, no request sent
00:04:04     result: FAILED (/eateries/1/menu)
00:04:04 --- menu read (expect: still blocked)
00:04:04 GET /eateries/1/menu BLOCKED: breaker is open, no request sent
00:04:04     result: FAILED (/eateries/1/menu)
00:04:04 === Phase 3: remove fault, wait out the recovery period
00:04:04 [demo] menu delay set to 0 ms
00:04:15 --- trial call (half-open)
00:04:15 [breaker] open -> half_open
00:04:15 [breaker] half_open -> closed
00:04:15 GET /eateries/1/menu -> 200 (attempt 1)
00:04:15     result: 200, 4 items
00:04:15 --- menu read (breaker closed again)
00:04:15 GET /eateries/1/menu -> 200 (attempt 1)
00:04:15     result: 200, 4 items
```

What the output shows

Behaviour	Where
Success	Phase 1, all calls `200`/`201`
Timeout and retry with backoff	Phase 2, attempts 1 and 2 fail with `ReadTimeout`, retries after 0.52 s and 1.08 s
Breaker opens	`[breaker] closed -> open` on the third failed attempt
Calls blocked while open	Two `BLOCKED` lines with no request sent to the server
Recovery	After the 10 s wait: `open -> half_open`, trial call succeeds, `half_open -> closed`, next call succeeds

Note: the breaker counts failed *attempts*, not whole calls, so it opened during the first failing call. The breaker's state-change line prints just before the caller's result line, which is why `closed -> open` appears above

`attempt 3/3 failed`.

---

6. OpenAPI

Export and validation

```
$ curl.exe -s localhost:8000/openapi.json -o openapi.json
$ Select-String -Path openapi.json -Pattern "HTTPValidationError" -Quiet
False
$ python -m openapi_spec_validator openapi.json
openapi.json: OK
```

`False` means the spec no longer contains FastAPI's default validation-error schema, so every 422 is documented with the project's own `ErrorResponse`.

Using the spec to test an endpoint

In Swagger UI (`http://localhost:8000/docs`, generated from the spec), I used "Try it out" on `POST /holds` with a body that breaks the documented constraint `item_id > 0`:

Request body:

```
{"item_id": 0}
```

Response `422`:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "request validation failed",
    "request_id": "6d311847750945c5",
    "details": [
      {
        "field": "item_id",
        "message": "Input should be greater than 0"
      }
    ]
  }
}
```

The response matches the `ErrorResponse` schema that the spec lists for the 422 response of `POST /holds` (`error.code`, `error.message`, `error.request_id`, `error.details`), so the documentation and the running API agree.

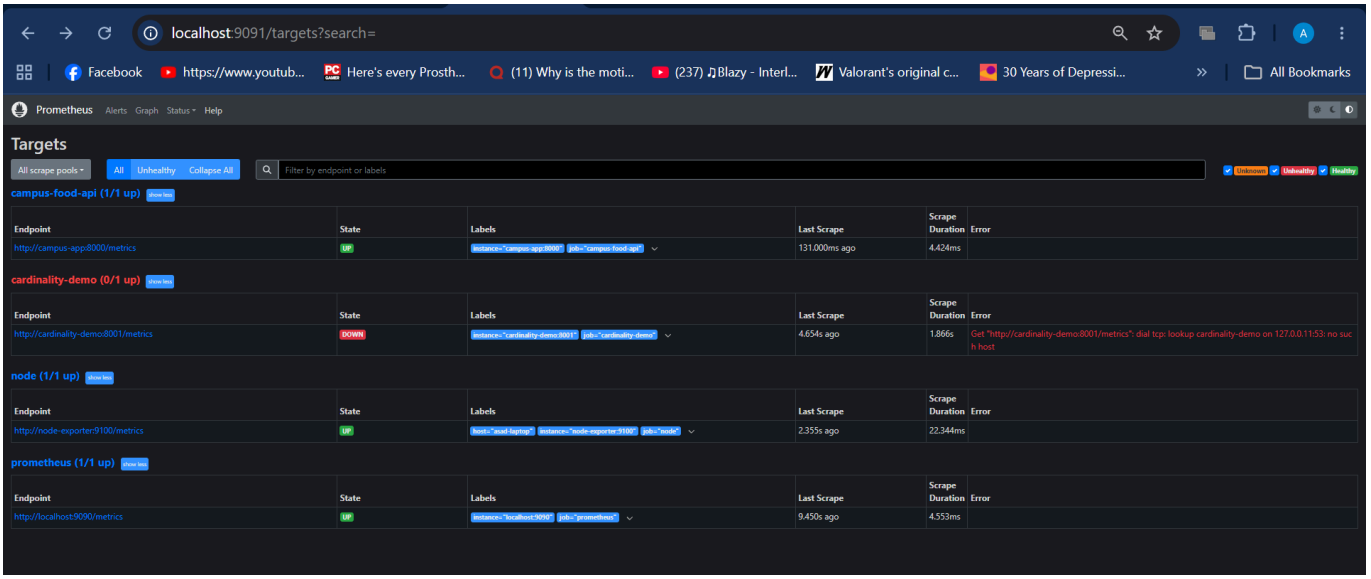
Code	Details
422	Error: Unprocessable Entity **Response body**

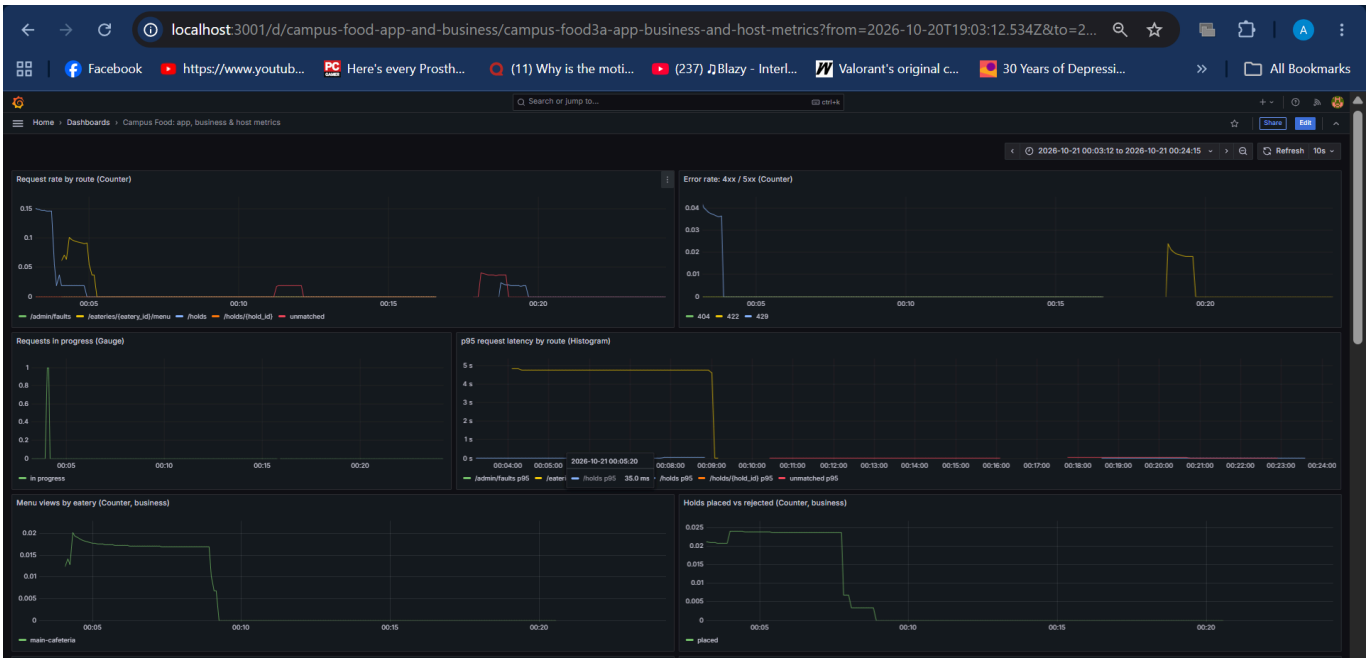

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "request validation failed",
    "request_id": "6d311847750945c5",
    "details": [
      {
        "field": "item_id",
        "message": "Input should be greater than 0"
      }
    ]
  }
}
```


---

7. A1 regression check

After the Part A changes (error envelope, validation, idempotency, rate limit), I re-ran the Assignment 1 traffic generator and checked Prometheus, Grafana and Kibana.










```
$ python -m traffic
tag=run requests=595 duration=120s
status codes: {201: 42, 200: 508, 409: 8, 429: 37}
  menu     n= 346 mean=    9.6 ms  p95=   26.5 ms
  item     n=  96 mean=    8.4 ms  p95=   21.1 ms
  hold     n=  87 mean=   12.2 ms  p95=   26.5 ms
  pickup   n=  25 mean=   17.2 ms  p95=   31.5 ms
  cancel   n=  15 mean=   14.8 ms  p95=   36.5 ms
  staff    n=  26 mean=   13.9 ms  p95=   25.4 ms
```

The rate limit was at its default (5 per 10 s) during this run. All 37 `429` responses came from hold creation (42 + 8 + 37 = 87 = the `hold` count); every menu, item, pickup, cancel and staff call succeeded. The 8 `409` responses are items that ran out of portions. This confirms the A1 traffic generator still works and that the limiter only affects `POST /holds`.

Tool	Result	Evidence
Prometheus (`localhost:9091/targets`)	`api` target UP. `cardinality-demo` is DOWN, which is expected: that container belongs to Experiment E2 and only starts with the `experiments` profile.	`docs/img/prometheus-targets.png`
Grafana (`localhost:3001`)	Dashboard panels show the request, hold and status-code activity from the run above.	`http://localhost:3000/goto/FHnVn0CDR?orgId=1`
Kibana (`localhost:5602`, data view `Campus food logs`)	Fresh log entries are indexed and each carries the `request_id`.	`![alt text](kibana_request_72542927ffbd49aa.png)`